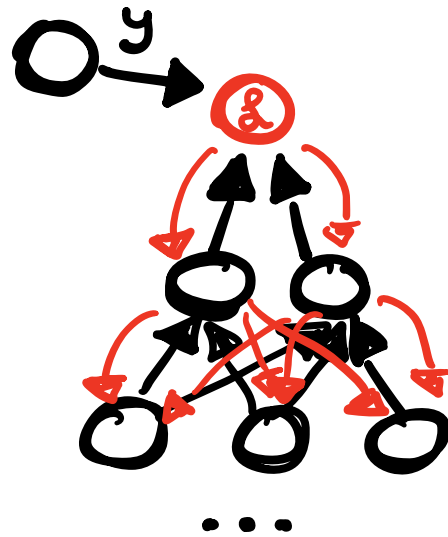


DS 4440

Optimizer Matters



Backprop yields gradients for all parameters efficiently ($\nabla_W L$).

Cool! ... Why did we want these again? For Gradient Descent!

$$\hat{W}_{\tau+1} \leftarrow \hat{W}_{\tau} - \underset{\substack{\text{learning} \\ \text{rate}}}{\alpha} \nabla_W L(\hat{W}_{\tau}, x, y)$$

Variants

Stochastic 1 instance (x, y) per step

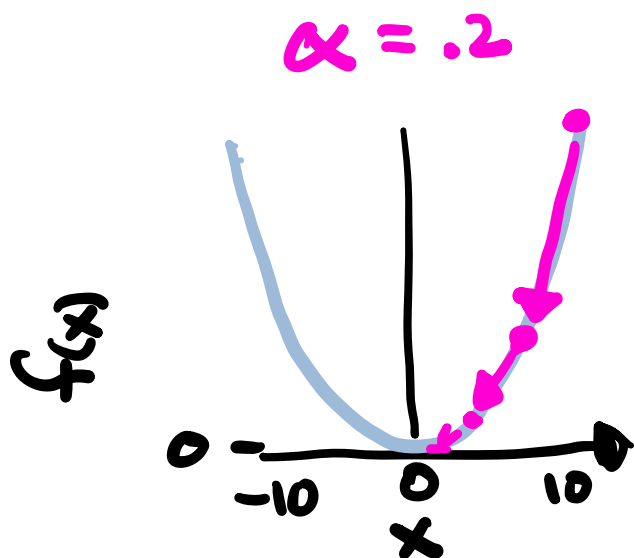
Batch Sets of (x, y) ; $\frac{1}{|B|} \sum_i \nabla_w \mathcal{L}_i$

Bigger Batch $\rightarrow$ Lower variance

The Importance of learning rate α

Consider a (very) simple example

$$f(x) = x^2 \quad \frac{d}{dx} f(x) = 2x \quad x_{init} = 10$$



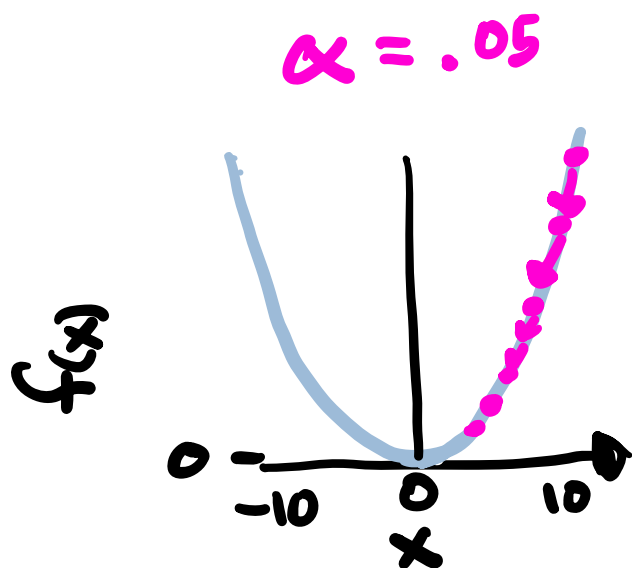
$$.2 \cdot (2 \cdot 10) = 4 \quad x = 6$$

$$.2 \cdot (2 \cdot 6) = 2.4 \quad x = 3.6$$

$$.2 \cdot (2 \cdot 3.6) = 1.44 \quad x = 2.16$$

...

But $\alpha = .05$



$$.05 \cdot (2 \cdot 10) = 1 \quad x=9$$

$$.05 \cdot (2 \cdot 9) = .9 \quad x=8.1$$

$$.05 \cdot (2 \cdot 8.1) = .81 \quad x \approx 7.3$$

... Slloooooow

Our update is:

$$x \leftarrow x - \alpha (2x) = \underbrace{(1 - 2\alpha)}_r x$$

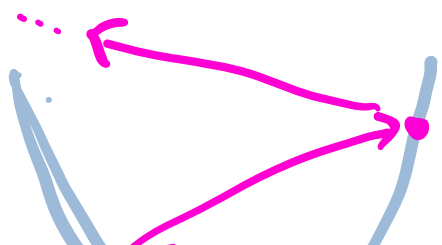
And after τ Steps:

$$x_\tau = r^\tau \cdot x_0.$$

So here we need $|r| < 1$ for Convergence.

$$\rightarrow |1 - 2\alpha| < 1 \rightarrow 0 < \alpha < 1$$

Consider: $\alpha = 1.2$

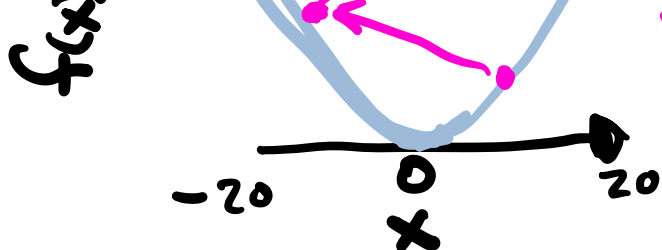


$$1.2 \cdot (2 \cdot 10) = 24$$

$$x = -14$$

$$1.2 \cdot (2 \cdot (-14)) = -33.6$$

$$x = 19.6$$



$$1.2 \cdot (2 \cdot 19.6) = 47.04 \quad x = -27.4$$

Diverges!

This is **Zig Zagging**. It happens because

Coordinates can be in Conflict. e.g.,

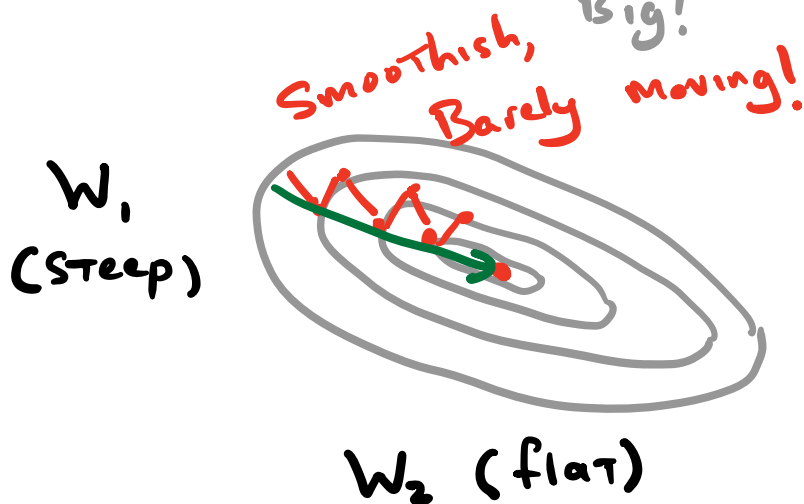
$$f(\omega_1, \omega_2) = \frac{1}{2}(a\omega_1^2 + b\omega_2^2) \quad \underline{a \gg b}$$

$$\omega_1 \leftarrow \underbrace{(1 - \alpha a)}_{r_1} \omega_1; \quad \omega_2 \leftarrow \underbrace{(1 - \alpha b)}_{r_2} \omega_2$$

Need $|r_1| < 1$ and $|r_2| < 1$. But $a \gg b$, and α needs to be $< \frac{2}{a}$ for r_1 .

But if $\alpha \approx \frac{1}{a}$, flatter direction only shrinks by factor of

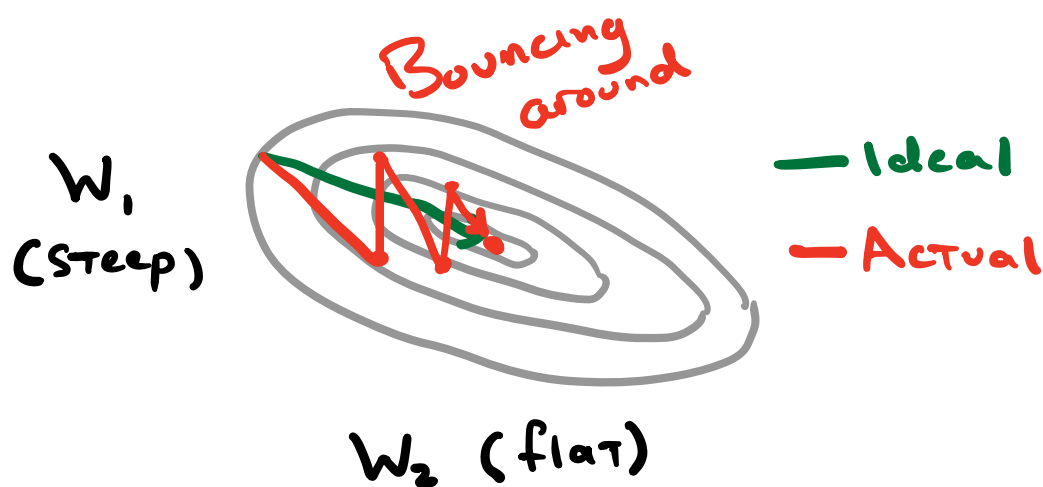
$$1 - \underbrace{\alpha b}_{\text{small!}} \approx 1 - \underbrace{\frac{b}{a}}_{\text{Big!}} = 1 - \underbrace{\frac{1}{k}}_{\text{close to 1}}; \quad \text{def } k = \frac{a}{b} \quad \text{"Condition number"}$$



— Ideal
— Actual

* $\alpha > 0$
otherwise
would be
ascent.

What if we crank up α ?



Normalizing inputs can help (and would fix it here, in this toy example.)

But this doesn't address issue in general (only touches inputs, e.g.).

Another general challenge:



Momentum

But adjust locally (e.g. .1)

$$v_t \leftarrow \gamma v_{t-1} - \alpha \nabla_{\hat{W}} \mathcal{L}(\hat{W}_t, x, y)$$

Keep going in direction
you've been going ($\delta = .9$, e.g.)

$$\hat{W}_{t+1} \leftarrow \hat{W}_t + v_t$$

Nesterov Momentum

We can even anticipate where
we are going:

$$v_t \leftarrow \delta v_{t-1} - \alpha \mathcal{L}(\underbrace{\hat{W}_t + \delta v_{t-1}}_{\text{Move toward accumulated } \nabla_w}, x, y)$$

Per parameter α s

So far we have assumed a
single learning rate (α) for
all parameters.

$$\text{Let } g_{t,j} = \nabla_w \mathcal{L}(\hat{W}_t)_j$$

$$\tilde{W}_{(\tau+1),j} \leftarrow W_{\tau,j} - \alpha_{\tau,j} g_{\tau,j}$$

AdaGrad

INTUITION Move slower for params

We have updated a bunch

$$\alpha_{\tau,j} \doteq \frac{\alpha}{\sqrt{G_{\tau,j}} + \epsilon} \text{ --- Avoids div. by } \emptyset.$$

Sum of grads for j.

$$\sum_{l \leq \tau} g_{l,j}^2$$

Cumulative updates slow rate.

RMSPprop Extends AdaGrad

by keeping a decaying average
of the squares of  terms.

$$S_{\tau,j} \doteq \lambda S_{(\tau-1),j} + (1-\lambda) g_{\tau,j}^2$$

Old grads fade fast

Then we divide α by $\sqrt{S_{\tau,j}}$.

Adam is maybe the go-to optimizer now. We keep two vars per param:

$$m_{\tau} \leftarrow \beta_1 m_{\tau-1} + (1-\beta_1) g_{\tau} \approx \text{Mom}_{\text{entum}}$$

$$v_{\tau} \leftarrow \beta_2 v_{\tau-1} + (1-\beta_2) g_{\tau}^2 \approx \text{RMS Prop}$$

Then

$$\hat{w}_{(\tau+1),j} \leftarrow \hat{w}_{\tau,j} - \frac{\alpha}{\sqrt{v_{\tau,j}} + \epsilon} \cdot m_{\tau,j}$$

Slightly Simplified; ignores bias-correction.

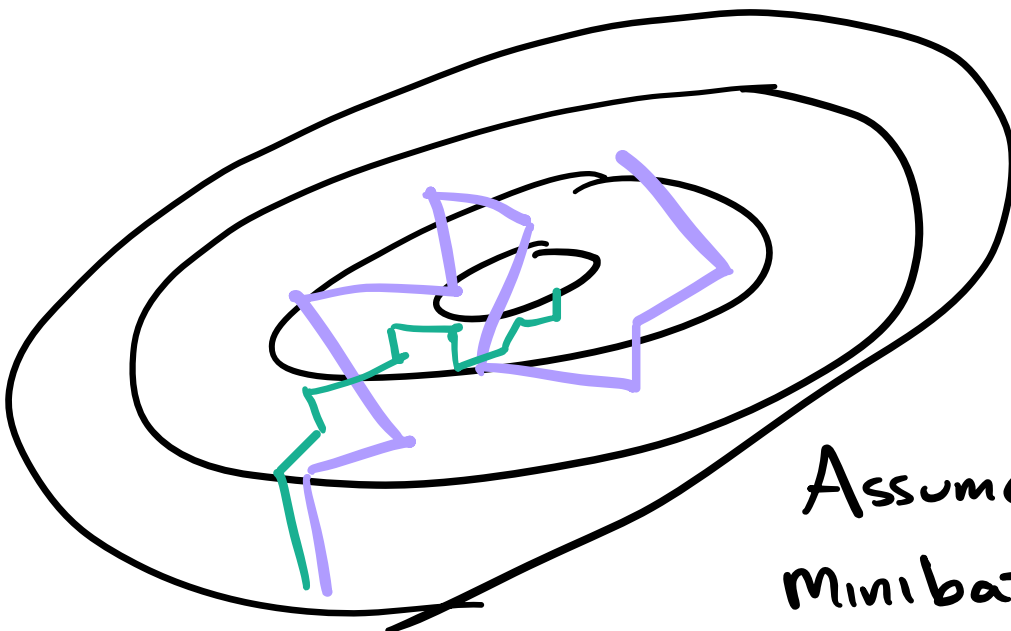
Optimizer recap

Optimizer	State	fixes
SGD	—	—
+ Momentum	v	Zig Zag
AdaGrad	G	per param rates
RMSProp	S	AdaGrad decaying to ϕ
Adam	m, v	both the above.

Learning rate decay

Loss Surface

Big Steps to
Start $\rightarrow$ Smaller
over Time



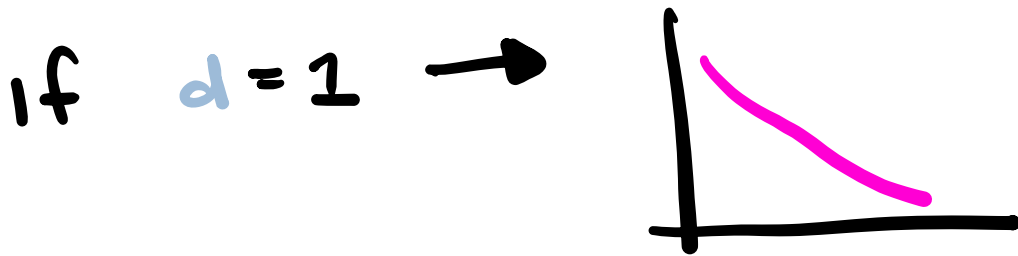
Assume noisy
minibatches

Big α might
Jump Too much

Shrinking α at each
Step might help

$$\alpha = \frac{1}{1 + d \cdot \text{epoch}} \cdot \alpha_0$$

decay rate



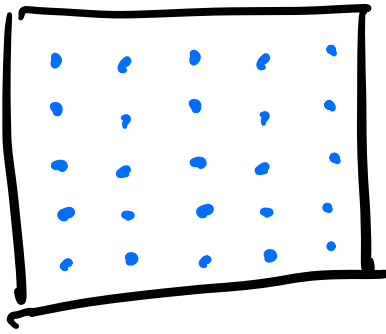
Can also do exponential decay

$$\alpha = (0.9)^{\text{epoch}} \cdot \alpha_0$$

Note on hyperparameter Tuning

- Choose random values (not grid)

Grid



Random

